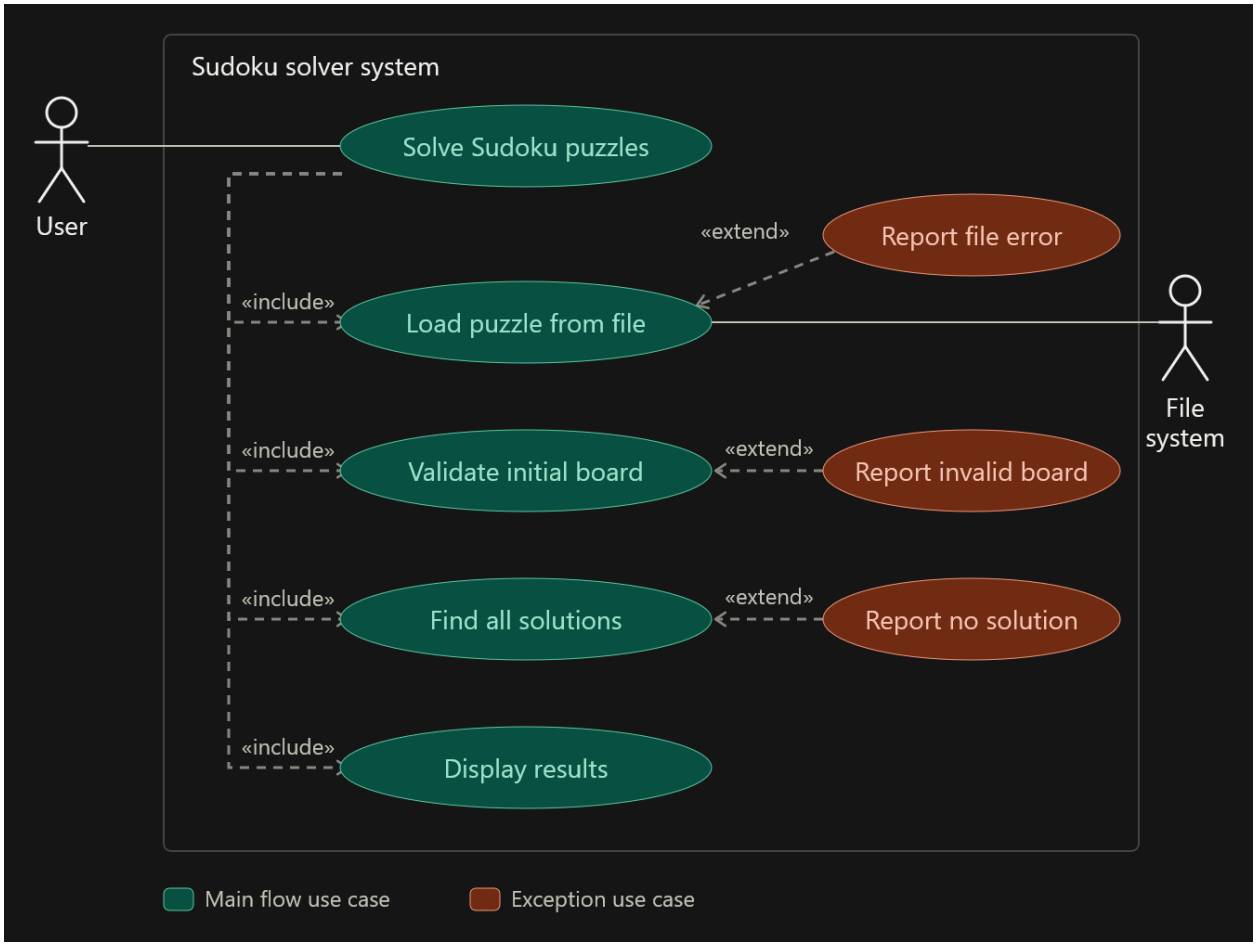


a. Use cases identified from the code

The code has two actors. The **User** runs the program from the command line and passes one or more puzzle filenames. The **File system** is a secondary actor that supplies the puzzle files.

ID	Use case	Where it comes from in the code
UC1	Solve Sudoku puzzles (primary)	`main()` loops over the filenames and calls `SudokuSolver::solve()`
UC2	Load puzzle from file	`Board::loadFromFile()`
UC3	Validate initial board	`Board::isInitialValid()` and `isValid()`
UC4	Find all solutions	`solveRecursive()` (backtracking)

UC5	Display results	`Board::print()` and the headers printed by `solve()`
UC6	Report file error (extends UC2)	The “Could not open file” and “Invalid file format” messages
UC7	Report invalid board (extends UC3)	The “Puzzle is invalid” message
UC8	Report no solution (extends UC4)	The “No solution found” message

UC1 includes UC2 to UC5. UC6 to UC8 are extensions that only run when something goes wrong.

b. Brief description (UC1: Solve Sudoku puzzles)

The user supplies one or more puzzle files on the command line. For each file, the system loads the 9×9 grid and checks it for rule violations. It then uses recursive backtracking to find every valid solution and prints the original puzzle and each solution. If a file can’t be read, the board breaks Sudoku rules, or no solution exists, the system reports it and moves on to the next file.

c. Step-by-step flow of events (outline)

Basic flow

1. The User runs the program with one or more puzzle filenames.
2. The system takes the next filename and creates a solver for it.
3. The system loads the puzzle from the file (UC2).
4. The system prints the file name and the original puzzle.
5. The system checks that the starting board has no duplicate digits in any row, column or 3×3 block (UC3).
6. The system searches for solutions (UC4). It tries digits 1–9 in each empty cell, backtracking on dead ends and continuing after each solution to find more.
7. The system prints each solution as “Solution #N” (UC5).
8. The system prints the closing border.
9. Steps 2–8 repeat for each remaining file, then the program exits normally.

Alternate and exception flows

- **1a. No filenames given.** The system prints the usage message and exits with code 1.
- **3a. File can’t be opened (UC6).** The system prints “Could not open file” and skips to the next file.
- **3b. File has fewer than 81 characters (UC6).** The system prints “Invalid file format” and skips to the next file.

- **5a. Starting board breaks the rules (UC7).** The system prints “Puzzle is invalid” and skips the search.
- **6a. No valid completion exists (UC8).** The system prints “No solution found”.
- **6b. Multiple solutions exist.** The system prints every solution, numbered in order.

d. Detailed description (UC1: Solve Sudoku puzzles)

Field	Description
Use case name	Solve Sudoku puzzles
Primary actor	User (command-line)
Secondary actor	File system (provides the puzzle files)
Goal	Obtain all valid solutions for each supplied Sudoku puzzle.
Trigger	The User runs `program <puzzle1.txt> [puzzle2.txt ...]`.
Preconditions	The program is compiled and run with at least one filename. Each puzzle file is a 9×9 grid of 81 non-whitespace characters. `_` marks a blank, and digits 1–9 are givens.
Postconditions (success)	The original puzzle and all of its solutions are on standard output. The program returns 0.
Postconditions (failure)	A message explains the problem (file error, invalid board or no solution). The puzzle is skipped and processing continues with the next file. With no arguments, the program exits with code 1.
Included use cases	Load puzzle from file, Validate initial board, Find all solutions, Display results
Extending use cases	Report file error, Report invalid board, Report no solution

Detailed behavior of the included use cases

- **Load puzzle from file (UC2).** The system reads characters one at a time and skips whitespace. It maps `_` to an empty cell (0) and 1–9 to their numeric values. Any other character is also treated as empty. If the file is missing or ends before 81 characters, loading fails and UC6 runs.
- **Validate initial board (UC3).** For every filled cell, the system checks that the same digit does not appear elsewhere in its row, column or 3×3 block. Any conflict makes the board invalid, and UC7 runs.
- **Find all solutions (UC4).** The system finds the first empty cell and tries each digit 1–9 that passes the validity check. It places the digit and recurses, then clears the cell on the

way back (backtracking). When no empty cell remains, it counts and prints a solution and keeps searching. If the count stays at 0, UC8 runs.

- **Display results (UC5).** The system prints a header with the file name, the original puzzle, and each numbered solution, with _ for any remaining blanks. Each puzzle's output ends with a border line.

Business rules

- Every row, column and 3×3 block must contain each digit 1–9 at most once.
- A solution exists only when all 81 cells are filled legally.
- One bad file never stops the run. Each file is handled independently.

Assumptions and notes

- The solver does not stop at the first solution, so a puzzle with many solutions (or a nearly empty grid) can take a very long time and print a very large amount of output.
- Rows and columns are not required to be on separate lines in the file, because whitespace is ignored.